IF25-11002 PRAKTIKUM PEMROGRAMAN

Pertemuan 13
Fungsi dan Modularitas

Memecah program menjadi bagian bernama: parameter, nilai kembali, prototipe, dan referensi

Tim Pengajar

Tahun 2026. Sejajar dengan Pertemuan 13 Algoritma Pemrograman: Fungsi dan Modularitas.

ITERA

DIKTISAINTEK BERDAMPAK

Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

SEBELUM MULAI

Dua mata kuliah, satu minggu

Topik minggu ini sama di dua mata kuliah, dengan peran yang berbeda.

Algoritma: Fungsi dan Modularitas

Dekomposisi masalah menjadi subprogram, prosedur dan fungsi, parameter masukan dan keluaran.

Praktikum: Fungsi

Mendefinisikan dan memanggil fungsi C++, memilih tipe kembali, memakai prototipe, pass by value dan by reference, serta array sebagai parameter.

TARGET HARI INI

Saat pulang nanti, kalian bisa

ITERA

1 Menerapkan

Menulis program yang dipecah menjadi fungsi dengan parameter dan nilai kembali yang tepat, termasuk fungsi `void`, fungsi `bool`, parameter referensi, dan parameter array

Sub-CPMK 13.1, CPMK0607

2 Menjelaskan

Menelusuri aliran nilai pada pemanggilan fungsi dan menjelaskan perbedaan pass by value dan pass by reference serta jangkauan variabel lokal

Sub-CPMK 13.2, CPMK0608

Dua kemampuan ini dinilai sama berat di Exit Ticket, tugas, UTS, dan UAS.

REVIEW, 5 MENIT

Ingat minggu lalu

Pertanyaan 1	Pertanyaan 2	Pertanyaan 3
Mengapa pertukaran butuh variabel sementara?	Elemen mana yang final sesudah satu putaran bubble sort?	Apa beda bubble sort dan selection sort?

Jawab di kertas dulu, lalu cocokkan dengan teman sebelah.

ITERA

Bagian 1

Masalah pembuka

Kode yang disalin berulang

MASALAH MINGGU INI

Kode yang disalin berulang

Program nilai kelas seorang asisten berisi kode mencari maksimum yang disalin empat kali: untuk kuis, tugas, UTS, dan UAS. Suatu hari ia menemukan kesalahan di kode itu.

Sekarang ia harus memperbaiki empat tempat, dan berharap tidak ada yang terlewat.

> **DISKUSI 2 MENIT**
>
> Bagaimana caranya menulis kode mencari maksimum cukup sekali, lalu memakainya untuk empat array berbeda?

Keluaran

```
Maks kuis  : 95
Maks tugas : 90
Maks UTS   : 88
Maks UAS   : 92
```

SAMBUNGAN DENGAN ALGORITMA

Dari langkah ke instruksi

ITERA

1 Temukan yang berulang

Mencari maksimum dilakukan empat kali dengan data berbeda.

2 Beri nama

Jadikan satu subprogram dengan masukan array dan keluaran nilai maksimum.

3 Terjemahkan

Satu fungsi `int maksimum(int a[], int n)`, dipanggil empat kali.

Langkah 1 dan 2 dilatih di Algoritma. Langkah 3 kita kerjakan hari ini dengan C++.

ITERA

Bagian 2

Coba dulu, baru dijelaskan

25 menit eksperimen di GDB Online

STRUGGLE, 25 MENIT

Misi kalian

ITERA

1 Namai

Pindahkan kode menampilkan garis `--------` ke luar `main` dan beri nama.

2 Beri masukan

Ubah agar panjang garis bisa diatur saat dipanggil.

3 Kembalikan

Buat bagian yang menerima dua bilangan dan memberikan yang terbesar.

Boleh

Bertanya ke teman, mengubah apa saja, sengaja membuat error.

Belum dulu

Mencari jawaban jadi di internet atau AI.

DEBRIEF STRUGGLE

Apa yang kalian temukan?

Pertanyaan 1	Pertanyaan 2	Pertanyaan 3
Di mana bagian bernama itu harus ditulis agar `main` mengenalnya?	Apa beda bagian yang menampilkan dan bagian yang memberikan nilai?	Apa yang terjadi pada variabel di dalam bagian itu setelah selesai?

Jawaban kalian akan kita uji di bagian berikutnya.

ITERA

Bagian 3

Memecah program menjadi fungsi

Definisi, parameter, nilai kembali, prototipe, dan referensi

Peta materi hari ini

Langkah	Yang kita bahas	Pertanyaan yang dijawab
1	Fungsi void dan pemanggilannya	Bagaimana memberi nama pada sekumpulan perintah agar bisa dipakai ulang?
2	Parameter dan nilai kembali	Bagaimana fungsi mengembalikan hasil ke pemanggil?
3	Prototipe fungsi	Bagaimana main tetap di atas tanpa compiler kebingungan?
4	Pass by value dan by reference	Kapan fungsi boleh mengubah variabel milik pemanggil?
5	Array sebagai parameter	Bagaimana mengirim array ke fungsi?
6	Variabel lokal dan jangkauannya	Sampai kapan sebuah variabel bisa dipakai?

LANGKAH 1 DARI 6: FUNGSI TANPA NILAI KEMBALI

Fungsi void dan pemanggilannya

Fungsi adalah sekumpulan perintah yang diberi nama supaya bisa dipanggil berulang kali. Fungsi `void` mengerjakan sesuatu, misalnya menampilkan, tanpa memberikan nilai kembali.

Saat `garis(10)` dipanggil, nilai 10 disalin ke parameter `n`, badan fungsi dijalankan, lalu eksekusi kembali ke baris sesudah pemanggilan.

> **CONTOH SEHARI-HARI**
>
> Fungsi seperti resep. Resep ditulis sekali di buku, lalu dipakai berkali-kali dengan bahan berbeda. Bahan yang dibawa ke dapur adalah argumen; daftar bahan di resep adalah parameter.

Bentuk umum

```
void nama(tipe parameter) {
    // perintah
}

nama(argumen);   // memanggil
```

Bagian yang ditulis miring di komentar diganti sesuai kebutuhan.

Fungsi void dan pemanggilannya: contoh

void.cpp

```cpp
#include <iostream>
using namespace std;
void garis(int n) {
    for (int i = 0; i < n; i++) cout << "-";
    cout << endl;
}
int main() {
    garis(10);
    cout << "Laporan Nilai" << endl;
    garis(10);
    return 0;
}
```

TEBAK DULU

Sebelum dijalankan, tulis keluarannya di kertas.

Fungsi void dan pemanggilannya: jawaban

ITERA

void.cpp

```cpp
#include <iostream>
using namespace std;
void garis(int n) {
    for (int i = 0; i < n; i++) cout << "-";
    cout << endl;
}
int main() {
    garis(10);
    cout << "Laporan Nilai" << endl;
    garis(10);
    return 0;
}
```

Keluaran

```
----------
Laporan Nilai
----------
```

`void` berarti fungsi tidak mengembalikan nilai. `n` adalah parameter; 10 adalah argumen.

Parameter dan nilai kembali

Fungsi dengan tipe selain `void` memberikan nilai kembali lewat `return`. Pemanggilannya bisa dipakai di mana pun nilai bertipe itu boleh dipakai: disimpan ke variabel, ditampilkan, atau menjadi argumen fungsi lain.

`return` langsung mengakhiri fungsi. Pastikan setiap jalur di dalam fungsi berakhir dengan `return`; bila ada jalur yang tidak, g++ memberi peringatan dan hasilnya tidak bisa ditebak.

CONTOH SEHARI-HARI

Seperti memesan jus di kantin: kita memberi pesanan (argumen), penjual mengerjakannya, lalu menyerahkan segelas jus (nilai kembali).

Bentuk umum

```
tipe_kembali nama(tipe p1, tipe p2) {
    return hasil;
}
int m = nama(a, b);
```

Bagian yang ditulis miring di komentar diganti sesuai kebutuhan.

Parameter dan nilai kembali: contoh

return.cpp

```cpp
double luasLingkaran(double r) {
    return 3.14 * r * r;
}
int lebihBesar(int a, int b) {
    return (a > b) ? a : b;
}
int main() {
    cout << luasLingkaran(10) << endl;
    int m = lebihBesar(7, 12);
    cout << m << " " << lebihBesar(m, 5) << endl;
    return 0;
}
```

TEBAK DULU

Sebelum dijalankan, tulis keluarannya di kertas.

Parameter dan nilai kembali: jawaban

return.cpp

```cpp
double luasLingkaran(double r) {
    return 3.14 * r * r;
}
int lebihBesar(int a, int b) {
    return (a > b) ? a : b;
}
int main() {
    cout << luasLingkaran(10) << endl;
    int m = lebihBesar(7, 12);
    cout << m << " " << lebihBesar(m, 5) << endl;
    return 0;
}
```

Keluaran

```
314
12 12
```

`return` mengirim nilai ke pemanggil dan langsung mengakhiri fungsi. Setiap jalur fungsi non-void harus berakhir dengan `return`.

LANGKAH 3 DARI 6: DEKLARASI DI ATAS MAIN

Prototipe fungsi

Compiler membaca dari atas ke bawah. Fungsi harus dikenalkan sebelum dipanggil. Ada dua cara: tulis definisinya di atas `main`, atau tulis prototipenya di atas dan definisinya di bawah. Prototipe membuat `main` tetap di atas dan mudah dibaca.

> **CONTOH SEHARI-HARI**
>
> Prototipe seperti daftar isi buku: memberi tahu bab apa saja yang ada, walaupun isinya baru muncul di halaman belakang.

Bentuk umum

```
bool genap(int x);   // prototipe, akhiri ;
int main() { ... genap(4) ... }
bool genap(int x) { ... }   // definisi
```

Bagian yang ditulis miring di komentar diganti sesuai kebutuhan.

Prototipe fungsi: contoh

prototipe.cpp

```cpp
#include <iostream>
using namespace std;
bool genap(int x);
int main() {
    for (int i = 1; i <= 4; i++) {
        bool g = genap(i);
        cout << i << (g ? " genap" : " ganjil") << endl;
    }
    return 0;
}
bool genap(int x) {
    return x % 2 == 0;
}
```

TEBAK DULU

Sebelum dijalankan, tulis keluarannya di kertas.

Prototipe fungsi: jawaban

prototipe.cpp

```cpp
#include <iostream>
using namespace std;
bool genap(int x);
int main() {
    for (int i = 1; i <= 4; i++) {
        bool g = genap(i);
        cout << i << (g ? " genap" : " ganjil") << endl;
    }
    return 0;
}
bool genap(int x) {
    return x % 2 == 0;
}
```

Keluaran

```
1 ganjil
2 genap
3 ganjil
4 genap
```

Prototipe memberi tahu compiler nama, tipe kembali, dan parameter fungsi, sehingga definisinya boleh di bawah `main`.

Pass by value dan by reference

Secara bawaan, argumen disalin ke parameter (pass by value). Perubahan di dalam fungsi hanya mengenai salinannya. Bila fungsi memang harus mengubah variabel pemanggil, misalnya fungsi tukar, gunakan parameter referensi dengan `&`.

CONTOH SEHARI-HARI

Memberi fotokopi dokumen kepada teman: coretan teman di fotokopi tidak mengubah dokumen asli kalian. Meminjamkan dokumen aslinya berbeda: coretannya langsung tertinggal di dokumen kalian.

Bentuk umum

```
void f(int x)    // salinan
void g(int &x)   // variabel aslinya
```

Bagian yang ditulis miring di komentar diganti sesuai kebutuhan.

Pass by value dan by reference: contoh

referensi.cpp

```
void tambahNilai(int x) { x += 10; }
void tambahRef(int &x) { x += 10; }

int main() {
    int a = 5, b = 5;
    tambahNilai(a);
    tambahRef(b);
    cout << a << " " << b << endl;
    return 0;
}
```

TEBAK DULU

Sebelum dijalankan, tulis keluarannya di kertas.

Pass by value dan by reference: jawaban

referensi.cpp

```cpp
void tambahNilai(int x) { x += 10; }
void tambahRef(int &x) { x += 10; }

int main() {
    int a = 5, b = 5;
    tambahNilai(a);
    tambahRef(b);
    cout << a << " " << b << endl;
    return 0;
}
```

Keluaran

```
5 15
```

Tanpa `&`, fungsi menerima salinan. Dengan `&`, fungsi bekerja langsung pada variabel aslinya.

Array sebagai parameter

Inilah jawaban masalah pembuka: satu fungsi `maksimum` dipakai untuk array berbeda. Fungsi tidak tahu panjang array, jadi panjangnya dikirim sebagai parameter kedua.

Berbeda dengan variabel biasa, array tidak disalin. Fungsi bekerja pada array aslinya, sehingga fungsi seperti `urutkan(a, n)` bisa mengubah isi array pemanggil.

CONTOH SEHARI-HARI

Seperti meminjamkan buku catatan asli ke teman, bukan fotokopinya: coretan teman langsung tertinggal di buku kita.

Bentuk umum

```
int maksimum(int a[], int n) { ... }
maksimum(data, 4);   // tanpa [ ]
```

Bagian yang ditulis miring di komentar diganti sesuai kebutuhan.

Array sebagai parameter: contoh

arrayparam.cpp

```cpp
int maksimum(int a[], int n) {
    int m = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > m) m = a[i];
    return m;
}
int main() {
    int kuis[4] = {80, 95, 70, 85};
    int uts[3] = {88, 76, 90};
    cout << maksimum(kuis, 4) << endl;
    cout << maksimum(uts, 3) << endl;
    return 0;
}
```

TEBAK DULU

Sebelum dijalankan, tulis keluarannya di kertas.

Array sebagai parameter: jawaban

arrayparam.cpp

```
int maksimum(int a[], int n) {
    int m = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > m) m = a[i];
    return m;
}
int main() {
    int kuis[4] = {80, 95, 70, 85};
    int uts[3] = {88, 76, 90};
    cout << maksimum(kuis, 4) << endl;
    cout << maksimum(uts, 3) << endl;
    return 0;
}
```

Keluaran

```
95
90
```

Panjang array dikirim lewat parameter terpisah. Perubahan elemen di dalam fungsi mengenai array aslinya.

Variabel lokal dan jangkauannya

Setiap variabel hanya bisa dipakai di bagian program tempat ia dideklarasikan (jangkauan atau scope). Variabel lokal sebuah fungsi lahir saat fungsi dipanggil dan hilang saat fungsi selesai. Karena itu dua fungsi boleh memakai nama variabel yang sama tanpa saling mengganggu.

> **CONTOH SEHARI-HARI**
>
> Seperti tamu acara: kartu tamu hanya berlaku di ruangan acara itu, dan tidak berlaku lagi setelah acara selesai.

Variabel lokal dan jangkauannya: ringkasan

Jenis	Dideklarasikan di	Hidup sampai	Contoh
Parameter	kurung fungsi	fungsi selesai	`n` pada `garis(int n)`
Lokal fungsi	badan fungsi	fungsi selesai	`m` pada `maksimum`
Lokal blok	di dalam `{ }`	blok selesai	`i` pada `for`

> Variabel `i` di `main` dan `i` di fungsi lain adalah dua variabel berbeda, walaupun namanya sama.

SEBELUM LATIHAN

Rangkuman materi

Konsep	Yang perlu diingat
Fungsi void dan pemanggilannya	`void` berarti fungsi tidak mengembalikan nilai.
Parameter dan nilai kembali	`return` mengirim nilai ke pemanggil dan langsung mengakhiri fungsi.
Prototipe fungsi	Prototipe memberi tahu compiler nama, tipe kembali, dan parameter fungsi, sehingga definisinya boleh di bawah `main`.
Pass by value dan by reference	Tanpa `&`, fungsi menerima salinan.
Array sebagai parameter	Panjang array dikirim lewat parameter terpisah.
Variabel lokal dan jangkauannya	Variabel `i` di `main` dan `i` di fungsi lain adalah dua variabel berbeda, walaupun namanya sama.

LATIHAN MENJELASKAN, 3 MENIT

Tebak keluarannya

tebak.cpp

```cpp
int f(int x, int &y) {
    x = x * 2;
    y = y + x;
    return x + y;
}

int main() {
    int a = 3, b = 4;
    int c = f(a, b);
    cout << a << " " << b << " " << c << endl;
    return 0;
}
```

Tulis keluarannya di kertas, karakter demi karakter. Jangan dijalankan dulu.

LATIHAN MENJELASKAN

Tebak keluarannya: jawaban

tebak.cpp

```cpp
int f(int x, int &y) {
    x = x * 2;
    y = y + x;
    return x + y;
}

int main() {
    int a = 3, b = 4;
    int c = f(a, b);
    cout << a << " " << b << " " << c << endl;
    return 0;
}
```

Keluaran

```
3 10 16
```

`x` salinan dari `a` menjadi 6, tetapi `a` tetap 3. `y` adalah referensi ke `b`, jadi `b` menjadi 4 + 6 = 10. Nilai kembali 6 + 10 = 16.

PESAN DIAMBIL LANGSUNG DARI G++

Error yang sering muncul minggu ini

Kesalahan	Contoh	Pesan pertama dari g++
Fungsi dipanggil sebelum dikenal	`panggil sebelum definisi`	`error: 'kuadrat' was not declared in this scope`
Tidak ada return di semua jalur	`int f() tanpa return akhir`	`warning: control reaches end of non-void function`
Banyak argumen tidak cocok	`lebihBesar(1, 2, 3)`	`error: too many arguments to function 'int lebihBesar(int, int)'`
Literal ke parameter referensi	`tukar(a, 5)`	`error: cannot bind non-const lvalue reference of type 'int&' to an rvalue of ...`
Nilai void dipakai	`int x = garis(5);`	`error: void value not ignored as it ought to be`

Pesan di tabel ini dihasilkan dengan g++ dan opsi -Wall, sama seperti di cek.sh.

Bagian 4

Hands-on bertingkat

45 menit, dari dasar sampai tantangan

Peta hands-on

No	File	Tingkat	Waktu	Yang dilatih
1	`handson1-bangun.cpp`	Dasar	10'	fungsi `void` dan fungsi dengan `return`
2	`handson2-prima.cpp`	Menengah	15'	fungsi `bool`, prototipe, array sebagai parameter
3	`handson3-referensi.cpp`	Tantangan	20'	telusuri parameter, perbaiki tiga kesalahan fungsi
+	`bonus-statistik.cpp`	Pengayaan	sisa	pustaka fungsi untuk array

Kerjakan berurutan. Cocokkan keluaran dengan folder `expected/`; latihan yang membaca masukan memakai data di folder `input/`.

HANDS-ON

Cara mengecek dan aturannya

ITERA

Di GDB Online

Salin isi file latihan, lengkapi TODO, klik Run. Bila program membaca masukan, ketik isi file di `input/`. Bandingkan dengan `expected/`.

Di komputer sendiri

Jalankan `bash cek.sh`. Skrip mengompilasi dengan `-Wall -Werror`, memberi masukan dari `input/`, lalu mencocokkan keluaran.

> AI boleh untuk memahami pesan error, tidak untuk meminta solusi utuh. Exit Ticket dikerjakan tanpa bantuan apa pun.

Bagian 5

Exit Ticket dan tugas

25 menit terakhir, lalu pekerjaan rumah

INDIVIDUAL, 25 MENIT, TANPA AI DAN INTERNET

Exit Ticket 13

Soal	Isi	Poin	CPMK
A1	Tulis fungsi yang mengembalikan nilai terbesar dari tiga bilangan	25	CPMK0607
A2	Tulis fungsi `void` yang mengubah array menjadi kuadrat setiap elemennya	25	CPMK0607
B1	Telusuri nilai variabel sebelum dan sesudah pemanggilan fungsi dengan referensi	20	CPMK0608
B2	Jelaskan mengapa sebuah fungsi tukar tidak bekerja	20	CPMK0608
B3	Jelaskan jangkauan variabel pada potongan kode yang diberikan	10	CPMK0608

Soal A dinilai dari keluaran yang tepat sama. Soal B dinilai dari ketepatan dan alasan.

DIKUMPULKAN SEBELUM PERTEMUAN 14

Tugas 13: Library Matematika

ITERA

Bagian A, program, 50 poin

Buat program berisi kumpulan fungsi matematika berikut dengan prototipe di atas `main`, lalu buat menu interaktif untuk memilih dan menguji setiap fungsi dengan validasi masukan. Ketentuannya di slide berikut.

Bagian B, penjelasan, 50 poin

Komentar maksimal 150 kata: telusuri pemanggilan `lcm(4, 6)` sampai ke `gcd`, jelaskan kapan memakai `void` dan kapan memakai tipe kembali, dan beri satu contoh perbedaan pass by value dan pass by reference dari program kalian.

Ketentuan Tugas 13

No	Ketentuan Bagian A
1	`int faktorial(int n)` dan `int pangkat(int basis, int eksponen)`
2	`bool isPrima(int n)`
3	`int gcd(int a, int b)` dan `int lcm(int a, int b)`
4	`double luasSegitiga(double alas, double tinggi)` dan `double kelilingLingkaran(double r)`
5	`void tampilkanDeret(int n)` yang mencetak 1 sampai n
6	Menu berulang dengan pilihan keluar
Bonus	Tambahkan fungsi `void tukar(int &a, int &b)` dan tunjukkan bedanya dengan versi tanpa `&`.

EMPAT LEVEL, SAMA DI SEMUA INSTRUMEN

Rubrik Tugas 13

Kriteria	Poin	Sangat baik 85–100	Baik 70–84	Cukup 55–69	Kurang <55
A1 Program berjalan	20	tanpa error dan peringatan	ada peringatan	perlu satu perbaikan	tidak terkompilasi
A2 Kelengkapan fungsi	15	delapan fungsi dan menu	satu fungsi kurang	dua kurang	tiga atau lebih kurang
A3 Ketepatan dan validasi	15	semua hasil tepat dan masukan divalidasi	satu salah	dua salah	sebagian besar salah
B1 Penelusuran pemanggilan	30	jejak lcm ke gcd tepat, alasan jelas	satu langkah keliru	tanpa jejak	tidak ada atau disalin
B2 Cerita error dan pengujian	20	pesan, sebab, perbaikan, uji	pesan tidak dikutip	hanya perbaikan	tidak ada

Nilai kriteria = poin × skor level. Bagian A total 50, Bagian B total 50.

PERSIAPAN

Sebelum Pertemuan 14

ITERA

1 Selesaikan

Hands-on yang belum lulus `cek.sh`, terutama latihan tantangan.

2 Kumpulkan

Tugas 13 sebelum Pertemuan 14 dimulai.

3 Baca

Modul Belajar 13 untuk mengulang, lalu bagian awal modul berikutnya.

Pertemuan 14 membahas rekursi, yaitu fungsi yang memanggil dirinya sendiri, dan struct untuk mengelompokkan data berbeda tipe.

ITERA

Terima kasih

Sampai jumpa di Pertemuan 14: rekursi dan struct